

Phase 2 Notes: BST Property and Searching

1. Main Goal of Phase 2

Phase 2 teaches how a **Binary Search Tree**, also called a **BST**, uses a special ordering rule to make searching faster.

In Phase 1, you learned about a normal binary tree. You learned that a tree is made of nodes, and each node has:

```
struct Node {  
    struct Node *lchild;  
    int data;  
    struct Node *rchild;  
};
```

You also learned these ideas:

- `root` is the top node of the tree.
- `lchild` points to the left child.
- `rchild` points to the right child.
- `NULL` means there is no node in that direction.
- A leaf node has no children.
- Recursion works well with trees.
- `count()` counts all nodes.
- `height()` finds the longest path downward.

Phase 2 builds on this foundation.

The main new idea is:

A normal binary tree becomes a Binary Search Tree when it follows a sorting rule.

By the end of Phase 2, you should understand:

- What a Binary Search Tree is.
- The BST ordering rule.
- Why BST searching can be faster than normal binary tree searching.
- What `key` means.
- What the current pointer `t` means.
- How successful search works.
- How unsuccessful search works.
- How recursive BST search works.
- How iterative BST search works.

- What tail recursion means in this topic.
 - Why iterative search uses less extra memory.
 - Why BST search depends on tree height.
 - Why search is $O(h)$ in general.
 - Why balanced BST search is $O(\log n)$.
 - Why skewed BST search can become $O(n)$.
-

2. Quick Review from Phase 1

Before learning BST searching, remember these Phase 1 ideas.

2.1 A Binary Tree Node

A binary tree node stores three things:

```
struct Node {  
    struct Node *lchild;  
    int data;  
    struct Node *rchild;  
};
```

Meaning:

- `lchild` stores the address of the left child.
- `data` stores the value inside the node.
- `rchild` stores the address of the right child.

Example node:

```
    30  
   /  
left right
```

The value `30` is stored in the `data` field.

2.2 Meaning of NULL

`NULL` means:

There is no node here.

Example:

```
  30
 /
20
```

Here:

```
30 has a left child: 20
30 has no right child
```

So:

```
root->lchild != NULL
root->rchild == NULL
```

This is very important in searching because if search reaches `NULL`, the value was not found.

3. What Is a Binary Search Tree?

A **Binary Search Tree** is a binary tree with a special rule.

The rule is:

```
For every node:
All smaller values go to the left.
All larger values go to the right.
```

Another way to write the rule is:

```
left subtree values < node value < right subtree values
```

This rule must be true for every node in the tree.

4. BST Rule in Simple Words

Imagine you are standing on a node.

If the value you are looking at is smaller than the current node, it must be somewhere on the left side.

If the value is larger than the current node, it must be somewhere on the right side.

So the BST rule is:

Small values go left.
Big values go right.

Example:

```
    30
   /
  20  40
```

This is a valid BST because:

20 is smaller than 30, so it is on the left.
40 is larger than 30, so it is on the right.

5. The BST Rule Must Be True at Every Node

A beginner may think the BST rule only needs to be true at the root.

That is wrong.

The rule must be true at every node.

Example:

```
    30
   /
  20  40
    /
   25
```

This is a valid BST.

Why?

At node 30:

20 and 25 are smaller than 30, so they are on the left side of 30.
40 is larger than 30, so it is on the right side of 30.

At node 20:

25 is larger than 20, so it is on the right side of 20.

So 25 is correct because:

25 > 20
25 < 30

That means 25 belongs:

inside the right subtree of 20
but still inside the left subtree of 30

6. Example of a Tree That Is Not a BST

This is a binary tree:

```
      30
     /
    40  20
```

But it is not a Binary Search Tree.

Why?

Because:

40 is greater than 30, but it is on the left side.
20 is smaller than 30, but it is on the right side.

So this tree breaks the BST rule.

Important idea:

Every BST is a binary tree, but not every binary tree is a BST.

7. Important Terms in Phase 2

7.1 Key

The **key** is the value you are searching for.

Example:

```
Search for 25
```

Here:

```
key = 25
```

7.2 Current Pointer `t`

In BST search code, we often use a pointer named `t`.

```
struct Node *t = root;
```

`t` means:

The current node we are standing on during the search.

At the beginning:

```
t points to root
```

As search continues, `t` moves left or right.

7.3 Dead End

A dead end happens when `t` becomes `NULL`.

```
t == NULL
```

This means:

We followed the correct search path, but there is no node there.

So the key is not in the tree.

8. Why BST Searching Is Faster Than Normal Binary Tree Searching

In a normal binary tree, there is no sorting rule.

So if you search for a value, you may need to check many nodes or even all nodes.

But in a BST, the ordering rule helps you choose one direction.

Example:

```
      30
     /
    20  40
     /
    25
```

If you are searching for `25`, start at `30`.

Compare:

`25 < 30`

So `25` cannot be on the right side of `30`.

That means you can ignore this whole part:

`40`

You only go left.

This is the power of BST search:

One comparison can remove an entire subtree from the search.

9. General BST Search Rule

To search in a BST:

```
Start at the root.  
  
If key == current node data:  
    the key is found.  
  
If key < current node data:  
    move to the left child.  
  
If key > current node data:  
    move to the right child.  
  
If the current pointer becomes NULL:  
    the key is not found.
```

In short:

```
Equal    -> found  
Smaller  -> go left  
Larger   -> go right  
NULL     -> not found
```

10. Search Algorithm in Plain English

Suppose we are searching for a key.

Step-by-step:

1. Start from the root node.
 2. Compare the key with the current node's data.
 3. If both are equal, stop. The search is successful.
 4. If the key is smaller, move to the left child.
 5. If the key is larger, move to the right child.
 6. Repeat the same process.
 7. If you reach `NULL`, stop. The key is not present.
-

11. Example Tree for Phase 2

We will use this tree:

```
      30
     /
    20  40
     \
      25
```

The connections are:

```
30 has left child 20
30 has right child 40
20 has right child 25
```

This is a valid BST.

Why?

```
20 < 30
40 > 30
25 > 20
25 < 30
```

12. Dry Run 1: Successful Search for 25

We search for:

```
key = 25
```

Tree:

```
      30
     /
    20  40
```

25

Step 1

Start at the root.

```
Current node = 30  
key = 25
```

Compare:

```
25 < 30
```

So move left.

```
t = t->lchild;
```

Now `t` points to 20.

Step 2

Current node is 20.

```
Current node = 20  
key = 25
```

Compare:

```
25 > 20
```

So move right.

```
t = t->rchild;
```

Now `t` points to 25.

Step 3

Current node is 25.

```
Current node = 25  
key = 25
```

Compare:

```
25 == 25
```

The key is found.

The function returns the address of node 25.

Final result:

```
25 found
```

13. Successful Search Table

Step	Current Node	Comparison	Move
1	30	$25 < 30$	Go left
2	20	$25 > 20$	Go right
3	25	$25 == 25$	Found

Result:

```
Search successful
```

14. Dry Run 2: Unsuccessful Search for 21

Now search for:

```
key = 21
```

Tree:

```
    30
   /
  20  40
   \
   25
```

Step 1

Start at the root.

```
Current node = 30
key = 21
```

Compare:

```
21 < 30
```

So move left.

```
t = t->lchild;
```

Now `t` points to `20`.

Step 2

Current node is `20`.

```
Current node = 20
key = 21
```

Compare:

```
21 > 20
```

So move right.

```
t = t->rchild;
```

Now `t` points to `25`.

Step 3

Current node is `25`.

```
Current node = 25  
key = 21
```

Compare:

```
21 < 25
```

So move left.

```
t = t->lchild;
```

But node `25` has no left child.

So now:

```
t = NULL
```

This means the search failed.

Final result:

```
21 not found
```

15. Unsuccessful Search Table

Step	Current Node	Comparison	Move
1	30	$21 < 30$	Go left
2	20	$21 > 20$	Go right
3	25	$21 < 25$	Go left
4	NULL	No node exists	Not found

Result:

Search unsuccessful

Important:

Reaching `NULL` during search is normal. It means the value is not in the tree.

16. Meaning of NULL During Search

When search reaches `NULL`, it means:

The value would have been here if it existed.
But this place is empty.

For example, when searching for `21`:

```
    30
   /
  20  40
   /
  25
 /
NULL
```

Since $21 < 25$, it should be on the left side of `25`.

But the left side of `25` is `NULL`.

So 21 is not in the tree.

17. Recursive BST Search

Recursive search uses a function that calls itself.

The function searches one node, then passes the job to either the left child or the right child.

Code

```
struct Node* Rsearch(struct Node *t, int key) {
    if (t == NULL) {
        return NULL;
    }

    if (key == t->data) {
        return t;
    } else if (key < t->data) {
        return Rsearch(t->lchild, key);
    } else {
        return Rsearch(t->rchild, key);
    }
}
```

18. Recursive Search: Function Header

```
struct Node* Rsearch(struct Node *t, int key)
```

This means:

The function returns a pointer to a Node.

It returns:

- the address of the found node, or
- NULL if the key is not found.

Parameters:

```
struct Node *t
```

`t` is the current node pointer.

```
int key
```

`key` is the value we are searching for.

19. Recursive Search: Base Case 1

```
if (t == NULL) {  
    return NULL;  
}
```

This means:

If there is no node here, the key is not found.

This is the unsuccessful search base case.

It stops recursion when the search reaches a dead end.

20. Recursive Search: Base Case 2

```
if (key == t->data) {  
    return t;  
}
```

This means:

If the key equals the current node's data, the search is successful.

The function returns `t`, which is the address of the current node.

21. Recursive Search: Move Left

```
else if (key < t->data) {  
    return Rsearch(t->lchild, key);  
}
```

This means:

If the key is smaller than the current node's data, search the left subtree.

The function calls itself using:

```
t->lchild
```

So the left child becomes the new current node.

22. Recursive Search: Move Right

```
else {  
    return Rsearch(t->rchild, key);  
}
```

This means:

If the key is larger than the current node's data, search the right subtree.

The function calls itself using:

```
t->rchild
```

So the right child becomes the new current node.

23. Why `return Rsearch(...)` Is Important

A common beginner mistake is writing:

```
Rsearch(t->lchild, key);
```

This calls the function, but it does not return the result to the previous caller.

Correct:

```
return Rsearch(t->lchild, key);
```

Why?

Because if the node is found deeper in the tree, that node address must travel back up through all recursive calls.

So always return the recursive call result.

24. Recursive Search Dry Run for 25

Tree:

```
    30
   /
 20  40
   /
  25
```

Call:

```
Rsearch(root, 25)
```

Call 1

```
t points to 30
key = 25
```

Compare:

```
25 < 30
```

Return:

```
return Rsearch(t->lchild, 25);
```

Now search moves to 20.

Call 2

```
t points to 20  
key = 25
```

Compare:

```
25 > 20
```

Return:

```
return Rsearch(t->rchild, 25);
```

Now search moves to 25.

Call 3

```
t points to 25  
key = 25
```

Compare:

```
25 == 25
```

Return:

```
return t;
```

The address of node 25 is returned.

Final result:

found

25. Recursive Search Dry Run for 21

Call:

Rsearch(root, 21)

Path:

30 -> 20 -> 25 -> NULL

Comparisons:

```
21 < 30, go left
21 > 20, go right
21 < 25, go left
NULL reached, return NULL
```

Final result:

not found

26. Tail Recursion

Recursive BST search is an example of **tail recursion**.

Tail recursion means:

The recursive call is the last major action done by the function.

Example:

```
return Rsearch(t->lchild, key);
```

After this recursive call returns, there is no extra calculation to do.

That makes recursive search easy to convert into a loop.

27. Why Tail Recursion Can Be Replaced by a Loop

In recursive search, the function keeps moving from one node to the next.

In iterative search, a loop does the same thing by updating `t`.

Recursive movement:

```
return Rsearch(t->lchild, key);
```

Iterative movement:

```
t = t->lchild;
```

Both mean:

```
Move to the left child and continue searching.
```

The loop version avoids extra recursive function calls.

28. Iterative BST Search

Iterative search uses a `while` loop instead of recursion.

Code

```
struct Node* search(struct Node *t, int key) {  
    while (t != NULL) {  
        if (key == t->data) {  
            return t;  
        } else if (key < t->data) {  
            t = t->lchild;  
        } else {  
            t = t->rchild;  
        }  
    }  
}
```

```
    }  
}  
  
return NULL;  
}
```

29. Iterative Search: Function Header

```
struct Node* search(struct Node *t, int key)
```

This function returns:

- a pointer to the found node, or
- `NULL` if the key is not found.

Parameters:

```
t = current node pointer  
key = value being searched
```

30. Iterative Search: Loop Condition

```
while (t != NULL)
```

This means:

```
Keep searching while the current pointer points to a real node.
```

If `t` becomes `NULL`, the loop stops.

That means the search failed.

31. Iterative Search: Match Found

```
if (key == t->data) {  
    return t;  
}
```

This means:

If the current node stores the key, return this node address.

32. Iterative Search: Move Left

```
else if (key < t->data) {  
    t = t->lchild;  
}
```

This means:

If the key is smaller, move t to the left child.

After this line, `t` points to a new node.

Future comparisons use the new `t`.

33. Iterative Search: Move Right

```
else {  
    t = t->rchild;  
}
```

This means:

If the key is larger, move t to the right child.

34. Iterative Search: Not Found

```
return NULL;
```

This line runs only after the loop ends.

The loop ends when:

```
t == NULL
```

So this means:

The search path ended at an empty place.
The key does not exist in the tree.

35. Iterative Search Dry Run for 25

Tree:

```
    30
   /
  20  40
   /
  25
```

Call:

```
search(root, 25)
```

At the start:

```
t = root
```

So `t` points to `30`.

Loop 1

```
t points to 30  
25 < 30
```

Move left:

```
t = t->lchild;
```

Now `t` points to `20`.

Loop 2

```
t points to 20  
25 > 20
```

Move right:

```
t = t->rchild;
```

Now `t` points to `25`.

Loop 3

```
t points to 25  
25 == 25
```

Return `t`.

Final result:

```
found
```

36. Iterative Search Dry Run for 21

Call:

```
search(root, 21)
```

Path:

```
30 -> 20 -> 25 -> NULL
```

Step-by-step:

```
21 < 30, move left  
21 > 20, move right  
21 < 25, move left  
left child of 25 is NULL
```

Now:

```
t == NULL
```

The loop stops.

The function returns:

```
return NULL;
```

Final result:

```
not found
```

37. Recursive Search vs Iterative Search

Both recursive and iterative search follow the same logic.

They both compare the key with the current node.

They both move:

```
left if the key is smaller
right if the key is larger
```

They both return:

```
node address if found
NULL if not found
```

The main difference is memory usage.

38. Comparison Table

Feature	Recursive Search	Iterative Search
Main technique	Function calls itself	Uses a while loop
Current node	Parameter <code>t</code>	Variable <code>t</code>
Move left	<code>Rsearch(t->lchild, key)</code>	<code>t = t->lchild</code>
Move right	<code>Rsearch(t->rchild, key)</code>	<code>t = t->rchild</code>
Found result	returns node pointer	returns node pointer
Not found result	returns <code>NULL</code>	returns <code>NULL</code>
Extra memory	Uses call stack	Uses constant memory
Space complexity	<code>O(h)</code>	<code>O(1)</code>

39. Why Recursive Search Uses $O(h)$ Space

Recursive search uses the function call stack.

Every time the function calls itself, one call is placed on the stack.

If the search path has height `h`, then there can be up to `h` active calls.

So recursive search uses:

```
O(h) extra space
```

Example path:

```
30 -> 20 -> 25
```

The stack may contain calls for:

```
Rsearch(30, 25)  
Rsearch(20, 25)  
Rsearch(25, 25)
```

That is why recursive search needs stack memory.

40. Why Iterative Search Uses $O(1)$ Space

Iterative search does not create new function calls.

It only updates one pointer:

```
t = t->lchild;
```

or:

```
t = t->rchild;
```

It keeps using the same variable `t`.

So iterative search uses:

```
 $O(1)$  extra space
```

`$O(1)$` means constant space.

The tree can be small or large, but the loop still mainly uses the same pointer variable.

41. Time Complexity of BST Search

BST search follows only one path from the root downward.

It does not visit every node.

So the time depends on the height of the tree.

General rule:

BST search time = $O(h)$

Here:

h = height of the tree

42. Why Search Time Is $O(h)$

At each level, search makes one comparison and moves one step down.

Example path:

30 -> 20 -> 25

This path has 3 visited nodes.

So the number of comparisons depends on how many levels the search path has.

That is why search time depends on height.

43. Balanced BST Search

A balanced BST has a small height.

Example:

```
    30
   /
  20  40
```

```
      /  \
    10   25  50
```

This tree is not very tall.

A search reaches values in only a few steps.

For a balanced BST:

h is about $\log n$

So search time becomes:

$O(\log n)$

This is the fast case.

44. Skewed BST Search

A skewed BST looks like a linked list.

Example:

```
10
  20
    30
      40
```

This is still a BST because every larger value is on the right.

But it is badly shaped.

To search for **40**, you must go through:

10 -> 20 -> 30 -> 40

That is like searching a linked list.

For a skewed BST:

`h can become n`

So search time becomes:

`O(n)`

This is the slow case.

45. Best Way to State BST Search Complexity

Do not say:

`BST search is always O(log n)`

That is not always true.

The correct general answer is:

`BST search is O(h)`

Then explain:

If the tree is balanced, h is about $\log n$, so search is $O(\log n)$.
If the tree is skewed, h can be n , so search is $O(n)$.

46. Complexity Table

Search Type	Time Complexity	Space Complexity	Reason
Recursive search	<code>O(h)</code>	<code>O(h)</code>	One recursive call per level

Search Type	Time Complexity	Space Complexity	Reason
Iterative search	$O(h)$	$O(1)$	One pointer moves down the tree
Balanced BST search	$O(\log n)$	Recursive: $O(\log n)$, Iterative: $O(1)$	Height is small
Skewed BST search	$O(n)$	Recursive: $O(n)$, Iterative: $O(1)$	Tree behaves like a linked list

47. Complete Phase 2 C Program

```

#include <stdio.h>
#include <stdlib.h>

struct Node {
    struct Node *lchild;
    int data;
    struct Node *rchild;
};

struct Node* createNode(int value) {
    struct Node *p = (struct Node*)malloc(sizeof(struct Node));

    if (p == NULL) {
        printf("Memory allocation failed\n");
        exit(1);
    }

    p->data = value;
    p->lchild = NULL;
    p->rchild = NULL;

    return p;
}

struct Node* Rsearch(struct Node *t, int key) {
    if (t == NULL) {
        return NULL;
    }

    if (key == t->data) {
        return t;
    }

```



```

    } else if (key < t->data) {
        return Rsearch(t->lchild, key);
    } else {
        return Rsearch(t->rchild, key);
    }
}

struct Node* search(struct Node *t, int key) {
    while (t != NULL) {
        if (key == t->data) {
            return t;
        } else if (key < t->data) {
            t = t->lchild;
        } else {
            t = t->rchild;
        }
    }

    return NULL;
}

int main() {
    struct Node *root = createNode(30);

    root->lchild = createNode(20);
    root->rchild = createNode(40);
    root->lchild->rchild = createNode(25);

    struct Node *a = search(root, 25);
    struct Node *b = search(root, 21);
    struct Node *c = Rsearch(root, 25);

    printf("Iterative search for 25: %s\n", a != NULL ? "found" : "not found");
    printf("Iterative search for 21: %s\n", b != NULL ? "found" : "not found");
    printf("Recursive search for 25: %s\n", c != NULL ? "found" : "not found");

    return 0;
}

```

48. Program Tree

The program creates this tree:

```
    30
   /
 20  40
   \
    25
```

These lines create the nodes:

```
struct Node *root = createNode(30);
root->lchild = createNode(20);
root->rchild = createNode(40);
root->lchild->rchild = createNode(25);
```

Meaning:

```
30 is the root.
20 is the left child of 30.
40 is the right child of 30.
25 is the right child of 20.
```

49. Program Output

Expected output:

```
Iterative search for 25: found
Iterative search for 21: not found
Recursive search for 25: found
```

Meaning:

```
25 exists in the tree.
21 does not exist in the tree.
Recursive search also finds 25.
```

50. Explanation of the Output

Output 1

```
Iterative search for 25: found
```

The iterative search follows:

```
30 -> 20 -> 25
```

So 25 is found.

Output 2

```
Iterative search for 21: not found
```

The iterative search follows:

```
30 -> 20 -> 25 -> NULL
```

So 21 is not found.

Output 3

```
Recursive search for 25: found
```

The recursive search also follows:

```
30 -> 20 -> 25
```

So 25 is found.

51. Common Beginner Mistakes

Mistake 1: Thinking Every Binary Tree Is a BST

Wrong idea:

If it has at most two children, it is a BST.

Correct idea:

If it has at most two children, it is a binary tree.
It is a BST only if it follows the ordering rule.

The BST rule is:

left values < node value < right values

Mistake 2: Checking Only the Root Rule

Some beginners check only the root.

But the BST rule must be true at every node.

Example:

```
      30
     /
    20  40
     /
    35
```

At first, this may look okay because:

20 < 30
40 > 30

But 35 is inside the left subtree of 30.

Since:

```
35 > 30
```

it should not be in the left subtree of 30.

So this is not a valid BST.

Mistake 3: Forgetting the NULL Check

Wrong:

```
if (key == t->data) {  
    return t;  
}
```

Problem:

If `t` is `NULL`, then `t->data` is invalid.

That can crash the program.

Correct:

```
if (t == NULL) {  
    return NULL;  
}  
  
if (key == t->data) {  
    return t;  
}
```

Always check `NULL` before using `t->data`.

Mistake 4: Forgetting to Return Recursive Calls

Wrong:

```
Rsearch(t->lchild, key);
```

Correct:

```
return Rsearch(t->lchild, key);
```

Why?

Because the found result must be passed back to the caller.

Mistake 5: Comparing with the Old Node After Moving

In iterative search:

```
t = t->lchild;
```

After this line, `t` points to a new node.

The next comparison must use the new `t`.

Do not mentally stay at the old node.

Mistake 6: Thinking `NULL` Means an Error

In search, `NULL` is not automatically an error.

It is the normal result for unsuccessful search.

`NULL` means:

```
The key was not found.
```

The error happens only if you try to access data through `NULL`, such as:

```
t->data
```

when:

```
t == NULL
```

Mistake 7: Saying BST Search Is Always $O(\log n)$

Wrong:

```
BST search is always  $O(\log n)$ .
```

Correct:

```
BST search is  $O(h)$ .
```

Then explain:

```
Balanced tree:  $O(\log n)$   
Skewed tree:  $O(n)$ 
```

52. Key Formulas and Rules to Remember

BST Rule

```
left subtree values < node value < right subtree values
```

Search Rule

```
key == node value -> found  
key < node value  -> go left  
key > node value  -> go right  
NULL              -> not found
```

General Time Complexity

```
BST search =  $O(h)$ 
```

Balanced Tree Search

$O(\log n)$

Skewed Tree Search

$O(n)$

Recursive Search Space

$O(h)$

Iterative Search Space

$O(1)$

53. Mini Practice 1

Given this BST:

```
      50
     /
    30  70
```

Search for:

30

Steps:

```
Start at 50.
30 < 50, go left.
Current node is 30.
30 == 30, found.
```


Result:

30 found

54. Mini Practice 2

Given this BST:

```
    50
   /
  30  70
```

Search for:

60

Steps:

```
Start at 50.
60 > 50, go right.
Current node is 70.
60 < 70, go left.
Left child of 70 is NULL.
```

Result:

60 not found

55. Mini Practice 3: Is This a BST?

Tree:

```
    10
   /
  5   20
```

This is a BST.

Why?

```
5 < 10
20 > 10
```

56. Mini Practice 4: Is This a BST?

Tree:

```
    10
   /
  15   20
```

This is not a BST.

Why?

```
15 > 10
```

But **15** is on the left side of **10**.

That breaks the BST rule.

57. Phase 2 Final Summary

Phase 2 teaches the first major operation in a Binary Search Tree: **searching**.

A BST is a binary tree with this ordering rule:

small values go left
large values go right

When searching, we compare the key with the current node.

If the key is equal to the current node's data, the search is successful.

If the key is smaller, we move left.

If the key is larger, we move right.

If we reach `NULL`, the search is unsuccessful.

Recursive search uses function calls.

Iterative search uses a loop.

Both follow the same search path.

Recursive search uses extra stack space:

$O(h)$

Iterative search uses constant extra space:

$O(1)$

BST search time depends on height:

$O(h)$

If the tree is balanced:

$O(\log n)$

If the tree is skewed:

$O(n)$

The biggest idea from Phase 2 is:

A BST does not search everywhere. It uses the ordering rule to choose only one path.

This prepares you for Phase 3, where insertion works like search first, then attaches the new node at the correct `NULL` position.

58. What You Should Be Able to Do After Phase 2

After studying Phase 2, you should be able to:

- Explain what a BST is.
 - State the BST ordering rule.
 - Check whether a small tree is a valid BST.
 - Explain what `key` means.
 - Explain what pointer `t` means.
 - Search for a value manually in a BST.
 - Explain successful search.
 - Explain unsuccessful search.
 - Understand why reaching `NULL` means not found.
 - Write recursive BST search.
 - Write iterative BST search.
 - Explain why recursive search uses stack space.
 - Explain why iterative search uses less memory.
 - Explain what tail recursion means in BST search.
 - State search complexity as $O(h)$.
 - Explain balanced BST search as $O(\log n)$.
 - Explain skewed BST search as $O(n)$.
 - Avoid the mistake of saying BST search is always $O(\log n)$.
-

59. Final Mental Model

Think of BST search like asking a question at every node:

Is this the value?

If yes:

Found.

If no, ask:

Is my key smaller or larger?

If smaller:

Go left.

If larger:

Go right.

If there is no node in that direction:

Not found.

That is the heart of Phase 2.